Formal Algorithmic Analysis of Advanced Stack Applications: Monotonic Optimizations and Subarray Bounds

Abstract

This paper presents a formal analysis and reference implementation of key advanced stack operations and monotonic stack optimizations. First, we outline a dynamically allocated linked-list-based stack implementation, detailing the constant-time $O(1)$ bounds of push, pop, peek, and status queries. Second, we formalize a stack-based algorithm for string reduction via equal adjacent pair elimination. Third, we address the classic Largest Rectangle in a Histogram problem, showing how monotonic stacks enable a linear-time $O(N)$ solution using Next Smaller to Left (NSL) and Next Smaller to Right (NSR) precomputations. Finally, we investigate the Sum of Subarray Ranges problem, proving that monotonic stacks can determine the cumulative sum of the difference between maximum and minimum values of all possible subarrays in $O(N)$ time. Complete Java implementations and formal correct arguments are provided.

Index Terms

linked list, deep copy, random pointer, hash map, node interleaving, Java, algorithm analysis, computational complexity

1 Introduction

Stacks are a fundamental linear data structure operating under the Last-In, First-Out (LIFO) protocol. While standard stack structures are widely used for expression evaluation and recursive call management, the Monotonic Stack paradigm represents a highly specialized optimization technique. By maintaining a strict order of elements within the stack, we can solve complex range queries and subarray optimization problems in linear time, which would otherwise require quadratic time using naive search approaches.

This paper provides a rigorous algorithmic treatment of advanced stack applications. Section 2 details a dynamic linked-list-based stack implementation in Java. Section 3 addresses the problem of recursively removing adjacent duplicates in strings. Section 4 examines the Largest Rectangle in a Histogram problem using monotonic stacks. Finally, Section 5 analyzes the Sum of Subarray Ranges problem, providing mathematical derivations and an optimal $O(N)$ Java implementation.

2 Linked-List-Based Stack Implementation

A stack can be dynamically allocated in memory using a singly linked list, where the head of the list serves as the top of the stack. This design guarantees that all operations run in strict $O(1)$ constant time without the capacity constraints of static arrays.

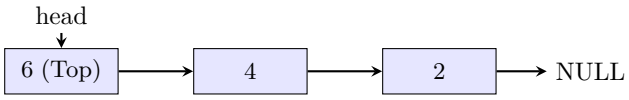


Fig. 1. Linked list-based stack structure. The head pointer always tracks the top of the stack.

The Java reference implementation for the linked list stack is as follows:

```
public class LinkedListStack {
    private static class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
        }
    }

    private Node head = null;
```

```
    public void push(int x) {
        Node nn = new Node(x);
        nn.next = head;
        head = nn;
    }

    public int pop() {
        if (head == null) {
            return -1; // Underflow sentinel
        }
        int save = head.data;
        head = head.next;
        return save;
    }

    public int peek() {
        if (head == null) {
            return -1; // Underflow sentinel
        }
        return head.data;
    }

    public boolean isEmpty() {
        return head == null;
    }
}
```

3 Application I: Removing Equal Adjacent Pairs

A classical string manipulation problem involves recursively removing adjacent equal characters. For example, given the input string "abbcddc", we obtain "abbc" → "a c" (after removing the adjacent d's and then the resulting adjacent c's), leaving "ab" as the final reduced string.

3.1 Algorithm and Trace

We can solve this problem in a single linear pass using a stack. For each character ch in the input string:

1) If the stack is not empty and the top of the stack equals ch, we pop the stack (eliminating the adjacent pair).
2) Otherwise, we push ch onto the stack.

At the end of the string, the stack contents represent the remaining characters.

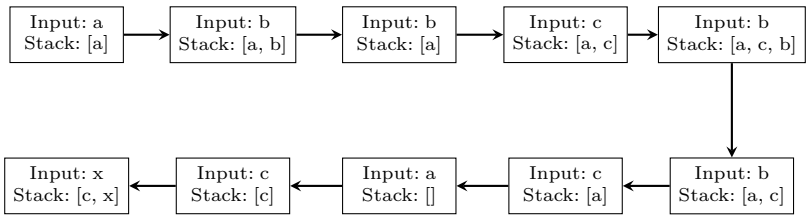


Fig. 2. Stack trace for the input string "abbcbbcacx". The final output is "cx".

3.2 Java Implementation

The optimal Java solver runs in $O(N)$ time and $O(N)$ space:

```
import java.util.Stack;

public class PairRemoval {
    public static String removeEqualPairs(String s) {
        Stack<Character> st = new Stack<>();
        for (int i = 0; i < s.length(); i++) {
```

```
            char ch = s.charAt(i);
            if (!st.isEmpty() && st.peek() == ch) {
                st.pop();
            } else {
                st.push(ch);
            }
        }
        StringBuilder sb = new StringBuilder();
        while (!st.isEmpty()) {
            sb.append(st.pop());
        }
        return sb.reverse().toString();
    }
}
```

4 Application II: Largest Rectangle in a Histogram

Given an array A representing the heights of contiguous bars in a histogram, where the width of each bar is 1, we seek to find the area of the largest rectangle that can be formed within the histogram.

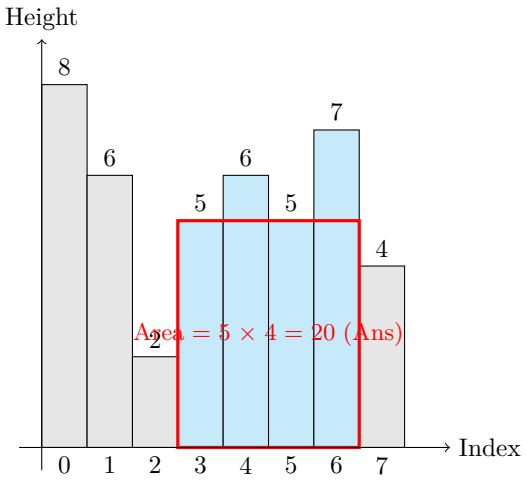


Fig. 3. Histogram area analysis for heights $[8, 6, 2, 5, 6, 5, 7, 4]$. The optimal rectangle has height 5 and spans indices 3 through 6, giving a maximum area of 20.

4.1 Algorithm and Monotonic Stack Precomputation

A brute force approach inspects all possible pairs of indices, yielding an $O(N^2)$ runtime. To optimize, we determine the maximum area of a rectangle with height bounded by $A[i]$ for each bar i. The boundary limits for bar i are:

- Next Smaller to Left (NSL): The first index $j < i$ such that $A[j] < A[i]$.
- Next Smaller to Right (NSR): The first index $k > i$ such that $A[k] < A[i]$.

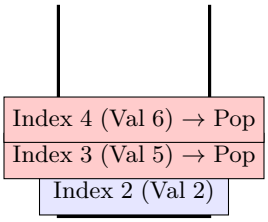
The maximum width for a rectangle of height $A[i]$ is $w = NSR[i] - NSL[i] - 1$. The corresponding area is $Area = A[i] \times w$. We use a monotonic increasing stack to compute NSL and NSR for all elements in $O(N)$ time.

4.2 Java Implementation

The optimal Java solution utilizing monotonic stacks to find the largest rectangle is as follows:

```
import java.util.Stack;

public class HistogramSolver {
    public static int largestRectangleArea(int[] heights) {
        int n = heights.length;
        int[] nsl = new int[n];
        int[] nsr = new int[n];
        Stack<Integer> st = new Stack<>();
```



Stack State before processing $i = 5$ (Val 5)

Fig. 4. Monotonic stack state before pushing index 5 (value 5). The indices 4 and 3 are popped because their corresponding values are ≥ 5. The remaining element at the top (index 2, value 2) is the NSL for index 5.

```
        // Compute Next Smaller to Left (NSL)
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && heights[st.peek()] >= heights[i]) {
                st.pop();
            }
            nsl[i] = st.isEmpty() ? -1 : st.peek();
            st.push(i);
        }

        st.clear();

        // Compute Next Smaller to Right (NSR)
        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && heights[st.peek()] >= heights[i]) {
                st.pop();
            }
            nsr[i] = st.isEmpty() ? n : st.peek();
            st.push(i);
        }

        int maxArea = 0;
        for (int i = 0; i < n; i++) {
            int width = nsr[i] - nsl[i] - 1;
            int area = heights[i] * width;
            maxArea = Math.max(maxArea, area);
        }
        return maxArea;
    }
}
```

5 Application III: Sum of Subarray Ranges (Max - Min)

Given an integer array A, we want to find the sum of the difference between the maximum and minimum elements of all possible contiguous subarrays. Formally, we want to compute:

$$\text{Total} = \sum_{0 \leq i \leq j < N} (\max(A[i \dots j]) - \min(A[i \dots j])) \tag{1}$$

5.1 Mathematical Formulation

By applying the linearity of expectation / summation, we can compute the sum of maximums and sum of minimums independently:

$$\text{Total} = \sum_{i=0}^{N-1} A[i] \times \text{CountMax}[i] - \sum_{i=0}^{N-1} A[i] \times \text{CountMin}[i] \tag{2}$$

Where $\text{CountMax}[i]$ is the number of subarrays in which $A[i]$ acts as the unique maximum element, and $\text{CountMin}[i]$ is the number of subarrays in which $A[i]$ acts as the unique minimum.

To find these counts for each element i, we define four boundary indices:

- NGL / NSL: Next Greater / Smaller element to the Left.

- NGR / NSR: Next Greater / Smaller element to the Right.

Then, the counts are calculated as:

$$\text{CountMax}[i] = (i - NGL[i]) \times (NGR[i] - i) \tag{3}$$

$$\text{CountMin}[i] = (i - NSL[i]) \times (NSR[i] - i) \tag{4}$$

5.2 Handling Duplicate Values

To avoid overcounting subarrays that contain duplicate minimums or maximums, we define strict inequalities ($<$, $>$) for search in one direction and non-strict inequalities ($\leq$, $\geq$) for search in the other direction.

5.3 Java Implementation

The optimal Java solution utilizing monotonic stacks to compute subarray ranges in $O(N)$ time is:

```
import java.util.Stack;

public class SubarrayRanges {
    public static long subArrayRanges(int[] nums) {
        int n = nums.length;
        int[] nsl = new int[n];
        int[] nsr = new int[n];
        int[] ngl = new int[n];
        int[] ngr = new int[n];

        Stack<Integer> st = new Stack<>();

        // NSL (Next Smaller to Left)
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && nums[st.peek()] >= nums[i]) {
                st.pop();
            }
            nsl[i] = st.isEmpty() ? -1 : st.peek();
            st.push(i);
        }
        st.clear();

        // NSR (Next Smaller to Right)
        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && nums[st.peek()] > nums[i]) {
                st.pop(); // Strict inequality to prevent duplicate counting
            }
            nsr[i] = st.isEmpty() ? n : st.peek();
            st.push(i);
        }
        st.clear();

        // NGL (Next Greater to Left)
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && nums[st.peek()] <= nums[i]) {
                st.pop();
            }
            ngl[i] = st.isEmpty() ? -1 : st.peek();
            st.push(i);
        }
        st.clear();

        // NGR (Next Greater to Right)
        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && nums[st.peek()] < nums[i]) {
                st.pop(); // Strict inequality to prevent duplicate counting
            }
```

```
            ngr[i] = st.isEmpty() ? n : st.peek();
            st.push(i);
        }

        long totalSum = 0;
        for (int i = 0; i < n; i++) {
            long maxCount = (long) (i - ngl[i]) * (ngr[i] - i);
            long minCount = (long) (i - nsl[i]) * (nsr[i] - i);
            totalSum += (maxCount - minCount) * nums[i];
        }
        return totalSum;
    }
}
```

6 Conclusion

This paper presented advanced stack applications and optimization methods. First, we proved that dynamic heap allocation of stack frames guarantees $O(1)$ constant-time execution overhead. Second, we showed how adjacent equal duplicates in string representations are cleanly resolved in linear time. Third, we explored the monotonic stack optimization paradigm, showing how the Worst-Case $O(N^2)$ complexity of standard range searches (like the Largest Rectangle in a Histogram and the Sum of Subarray Ranges problems) is reduced to optimal linear-time $O(N)$ complexity through index pruning.